



Group Members

Student 1: Gaurav Acharya — [MT26MCS024]
Student 2: Sandeep — [MT26MCS032]

Guide Name

Dr. A. S. Mokhade

Practical Title:

CSP502 Software Lab-I:
Practical 5: From Big-O to Performance

Problem Statement:

Reasoning About Algorithmic Growth Before Measuring It

Part A

The four hypothetical running-time functions are:

$$T_1(n) = 1000n$$

$$T_2(n) = 5n^2$$

$$T_3(n) = 50n \log_2 n$$

$$T_4(n) = n^2 + 500n + 10000$$

- **State the Big-O growth class that you would associate with each expression.**

$T_1(n) = 1000n$ is classified as $O(n)$.

$T_2(n) = 5n^2$ is classified as $O(n^2)$.

$T_3(n) = 50n \log_2 n$ is classified as $O(n \log n)$.

$T_4(n) = n^2 + 500n + 10000$ is classified as $O(n^2)$.

- **Which expressions have the same asymptotic growth class?**

Expressions $T_2(n)$ and $T_4(n)$ share the same asymptotic growth class, $O(n^2)$.

- **Arrange them according to their expected growth for sufficiently large n .**

Ordered from the slowest expected growth to the fastest, the sequence is: $T_1(n) < T_3(n) < T_4(n) < T_2(n)$.

Note: While both T_2 and T_4 belong to $O(n^2)$, T_2 has a larger leading coefficient (5 compared to 1). Therefore, as $n \rightarrow \infty$, $T_2(n)$ will strictly bound $T_4(n)$ from above.

- **Does this ordering necessarily tell you which expression has the smallest value for $n = 10$, $n = 100$, and $n = 10,000$?**

No, asymptotic ordering does not dictate which expression yields the smallest value for finite, relatively small inputs. Constant factors and lower-order terms heavily influence the output at these scales.

- **For $n = 10$:** $T_2(10) = 500$ produces the smallest value.
- **For $n = 100$:** $T_3(100) \approx 33,219$ produces the smallest value.
- **For $n = 10,000$:** $T_3(10,000) \approx 6,643,856$ produces the smallest value. (The linear function T_1 yields 10,000,000 and does not become the smallest until $n > 2^{20}$).

- **Can an expression belonging to $O(n^2)$ have a smaller value than one belonging to $O(n \log n)$ for some values of n ?**

Yes. An expression belonging to $O(n^2)$ can yield a smaller value than one belonging to $O(n \log n)$ for specific, small values of n . As demonstrated above, when $n = 10$, the $O(n^2)$ function $T_2(10) = 500$ evaluates lower than the $O(n \log n)$ function $T_3(10) \approx 1660$.

- **If yes, investigate approximately where the relative ordering changes.**

To find where the relative ordering between $T_2(n)$ and $T_3(n)$ permanently changes, the expressions are equated:

$$5n^2 = 50n \log_2 n$$

$$n = 10 \log_2 n$$

Solving this equation for n yields a crossover point of approximately $n \approx 58.77$. For all integer inputs $n \geq 59$, $T_2(n)$ will consistently output a larger value than $T_3(n)$.

- **What information present in the four expressions is not visible when we describe them only using Big-O?**

Big-O notation hides constant multipliers (e.g., the 1000 in T_1) and lower-order additive terms (e.g., the $500n + 10000$ in T_4).

Record one conclusion: From this exercise, what does Big-O preserve and what does it abstract away?

- **What Big-O Preserves:** It preserves the dominant scaling factor—the fundamental rate at which the resource requirements of an algorithm grow as the input size approaches infinity.
- **What Big-O Abstracts Away:** It abstracts away constant coefficients and less significant additive terms, under the mathematical assumption that these factors become insignificant relative to the highest-order term for sufficiently large inputs.

Part B

Fragment 1 Analysis

- **Measure of input size:** n , representing the limit of the loop.
- **Operation considered:** The `process(A[i])` function call.
- **Growth with input size:** The operation executes exactly n times, growing linearly with the input.
- **Asymptotic behaviour:** $O(n)$.
- **Required assumption:** The `process()` operation must execute in constant $O(1)$ time.

Fragment 2 Analysis

- **Measure of input size:** n .
- **Operation considered:** The `process(A[i], A[j])` function call.
- **Growth with input size:** The outer loop runs n times, and for each iteration, the inner loop runs n times. The operation executes $n \times n = n^2$ times.
- **Asymptotic behaviour:** $O(n^2)$.
- **Required assumption:** The `process()` operation executes in constant $O(1)$ time.

Fragment 3 Analysis

- **Measure of input size:** n .
- **Operation considered:** The `process(A[i])` function call and the assignment $i = 2 * i$.
- **Growth with input size:** The loop variable i doubles in each iteration (1, 2, 4, 8, ...). The loop terminates when $i \geq n$, meaning the operation executes approximately $\log_2 n$ times.
- **Asymptotic behaviour:** $O(\log n)$.
- **Required assumption:** The `process()` operation executes in constant $O(1)$ time.

Fragment 4 Analysis

- **Measure of input size:** n .
- **Operation considered:** The comparison `if A[i] == target`.
- **Growth with input size:** The growth depends on the location of the target. In the worst-case scenario, it executes n times.
- **Asymptotic behaviour:** $O(n)$.
- **Required assumption:** Array access and comparison operations execute in constant $O(1)$ time.

Questions for Fragment 4

- **Target at the beginning:** The loop finds the target on the first iteration and returns immediately, resulting in a constant execution time of $O(1)$.
- **Target near the end:** The loop must iterate through almost all elements, scaling linearly towards $O(n)$.
- **Target absent:** The loop evaluates every single element without success, strictly executing exactly n times, which is $O(n)$.
- **Conclusion on single complexity expressions:** Describing an algorithm with only one complexity expression typically only captures the worst-case bound. It fails to illustrate that algorithms can have significantly different best-case, average-case, and worst-case execution profiles depending on the specific properties of the input.

Fragment 5 Analysis

- **Measure of input size:** n .
- **Operation considered:** The `process(A[i])` call in the first loop and the `compare(A[j], A[k])` call in the nested loops.
- **Growth with input size:** The first loop executes n times. The independent nested loops execute n^2 times. Total operations scale proportionally to $n^2 + n$.
- **Asymptotic behaviour:** $O(n^2)$.
- **Required assumption:** Both the `process()` and `compare()` operations execute in constant $O(1)$ time.

Questions for Fragment 5

- **Which part determines asymptotic growth:** The nested j and k loops determine the asymptotic growth. As n grows sufficiently large, the n^2 term dominates the linear n term, making the linear portion mathematically insignificant in the limit.
- **Practical cost distinction:** Big-O abstracts away the $O(n)$ loop, but it does not mean the loop lacks a practical cost. In physical execution, running `process(A[i])` requires CPU cycles and memory access. For small values of n , or if `process()` contains a massive constant overhead hidden by $O(1)$, this “theoretically insignificant” portion of the code could account for a large percentage of the actual execution time.

Part C

Claim 1: Algorithm A is $O(n)$, whereas Algorithm B is $O(n^2)$. Therefore, Algorithm A will always execute faster.

- **Classification:** Unjustified.
- **Reasoning:** Big-O notation only describes asymptotic behavior as n approaches infinity. For small values of n , Algorithm B could execute faster if it has significantly smaller constant factors or lower-order terms.
- **Additional Information Required:** We need to know the typical input size (n) the program will handle in production and the exact time complexity expressions (including constants) to find the crossover point.

Claim 2: Two algorithms are both $O(n \log n)$. Their execution times should therefore be approximately equal.

- **Classification:** Unjustified.
- **Reasoning:** Two algorithms in the same complexity class can have vastly different execution times due to hidden constant multipliers and overhead. For instance, $2n \log n$ and $1000n \log n$ are both $O(n \log n)$, but the latter takes roughly 500 times longer to execute.
- **Additional Information Required:** We need to know the constant factors, memory access patterns, and the specific operations being performed by both algorithms.

Claim 3: When the input size was doubled, execution time approximately quadrupled. Therefore, the algorithm is $O(n^2)$.

- **Classification:** Conditionally Justified.
- **Reasoning:** A quadrupling of execution time upon doubling the input size is mathematically consistent with an $O(n^2)$ growth rate (since $(2n)^2 = 4n^2$). However, empirical timing alone does not mathematically prove the theoretical worst-case bound; the slowdown could be due to hardware bottlenecks, memory paging, or specific worst-case input configurations rather than pure algorithmic complexity.
- **Additional Information Required:** Code inspection or formal mathematical analysis of the algorithm's loops and operations is required to prove the theoretical bound.

Claim 4: An $O(n^2)$ algorithm should never be selected when an $O(n \log n)$ algorithm exists for the same problem.

- **Classification:** Unjustified.
- **Reasoning:** If the guaranteed input size is always very small, the $O(n^2)$ algorithm might actually be faster due to lower setup costs and constant overhead (e.g., Insertion Sort often beats Merge Sort for tiny arrays). Furthermore, $O(n^2)$ algorithms are sometimes chosen because they are simpler to implement, easier to maintain, or require less auxiliary memory space (in-place vs. out-of-place sorting).

- **Additional Information Required:** We need the constraints of the specific software engineering problem, including memory limits, expected input sizes, and development time budgets.

Claim 5: Program A completed in 0.8 seconds and Program B in 1.2 seconds. Therefore, Algorithm A is more efficient.

- **Classification:** Unjustified.
- **Reasoning:** Raw execution time is influenced by numerous factors outside of the algorithm itself, including compiler optimizations, programming language speed, hardware specifications, background operating system processes, and the specific data provided.
- **Additional Information Required:** We need to ensure a controlled experimental environment where both programs are written in the same language, compiled similarly, executed on the exact same hardware, and tested against identical and varied data sets.

Claim 6: Big-O tells us how an algorithm's resource requirement grows with input size; it does not tell us directly how many seconds a particular implementation will take.

- **Classification:** Justified.
- **Reasoning:** This is the precise definition and purpose of asymptotic notation. It provides a theoretical mathematical model for scaling, intentionally abstracting away hardware-specific timings and language overhead.

Dangerous Claims in Software Design

Most Dangerous Claim: **Claim 4** (An $O(n^2)$ algorithm should never be selected when an $O(n \log n)$ algorithm exists) or **Claim 1** (Algorithm A is always faster).

- **Why:** Accepting Claim 4 without qualification frequently leads to over-engineering. Developers might spend excessive time implementing, debugging, and maintaining a highly complex $O(n \log n)$ algorithm for a feature that only ever processes 20 items. This wastes development resources and can sometimes result in slower actual execution times due to the large constant overhead of the complex algorithm, harming overall system maintainability and performance.

Part D

Consider two hypothetical implementations whose running times approximately follow: $T_A(n) = 20n$ and $T_B(n) = 200n$.

- **1. What is the Big-O complexity of A and B?**

Both $T_A(n)$ and $T_B(n)$ belong to the $O(n)$ complexity class.

- **2. Does Big-O distinguish their actual amount of work?**

No. Big-O notation intentionally abstracts away constant coefficients (the 20 and the 200). Therefore, it cannot distinguish the actual amount of work or raw execution time between these two specific implementations.

- **3. If both algorithms process one million items, could the difference matter in a real system?**

Yes, absolutely. Processing $n = 1,000,000$ items would require roughly 20,000,000 operations for Algorithm A and 200,000,000 operations for Algorithm B. Algorithm B will take approximately 10 times longer to execute. In real-world systems, a 10x difference in performance can significantly impact user experience, server costs, or real-time processing constraints.

- **4. Does this make Big-O incorrect or incomplete for this particular question?**

It makes Big-O *incomplete* for answering questions about precise execution time, but not incorrect. Big-O perfectly describes how both algorithms scale (if input doubles, execution time doubles for both), but it lacks the necessary constant factors required to calculate absolute performance values.

Now consider: $T_C(n) = 2n^2$ and $T_D(n) = 100n \log_2 n$.

- **5. Which has the better asymptotic growth?**

$T_D(n) = 100n \log_2 n$ has the better asymptotic growth because the $O(n \log n)$ class grows much slower than the $O(n^2)$ class as n approaches infinity.

- **6. Must it therefore be faster for every n ?**

No. Because $T_D(n)$ has a much larger constant multiplier (100 compared to 2), $T_C(n)$ will actually evaluate to a smaller value (meaning it is faster) for small values of n .

- **7. Estimate or investigate a region in which their relative performance changes. What would you call such a region?**

To find the change, equate the two expressions:

$$2n^2 = 100n \log_2 n$$

$$n = 50 \log_2 n$$

Solving this iteratively or graphically shows that $n \approx 438$. For $n < 438$, $T_C(n)$ is faster. For $n \geq 438$, $T_D(n)$ permanently overtakes it.

This region or specific value is commonly referred to as the **crossover point** or intersection point. The domain before this point ($n < 438$) is often called the **small- n regime**.

Part E

Task 1: Classify the factors into two broad groups

Factors primarily related to the algorithm/implementation:

- Constant factors
- Lower-order terms
- Function-call overhead
- Characteristics of the input

Factors primarily related to the execution environment:

- Programming language
- Compiler and optimization
- Operating-system activity
- Hardware

Task 2: Analyse factors that cannot be cleanly placed into only one category

- **Memory allocation:** This bridges both categories. The frequency and size of allocations are dictated by the *algorithm/implementation*, but the actual time cost of finding and reserving that memory is heavily dependent on the *execution environment* (the OS memory manager and hardware speed).
- **Cache behaviour:** While CPU caching is fundamentally a *hardware/environment* mechanism, an algorithm's memory access pattern (its *implementation*) directly dictates whether cache hits or misses occur. Designing cache-friendly data structures requires considering both the algorithm and the physical machine architecture simultaneously.

Task 3: If Big-O abstracts away many of these factors, why is Big-O still useful?

Big-O notation remains an essential tool because it provides a universal, hardware-independent baseline for evaluating scalability. While execution environment factors might multiply an algorithm's running time by a constant (e.g., running 5x faster on a better CPU), they cannot change the fundamental shape of the growth curve.

When input sizes become sufficiently large, the mathematical growth rate (Big-O) will always eventually dominate all environmental optimizations. An $O(n \log n)$ algorithm running on an old, slow computer in a slow programming language will still eventually outpace an $O(n^2)$ algorithm running on a modern super-computer written in highly optimized assembly, simply because of how the resource requirements scale as $n \rightarrow \infty$. Big-O allows engineers to make robust, future-proof architectural decisions before writing a single line of code.

Part F

AI Comparison Table

Aspect	Our Group's Reasoning	AI's Reasoning	Agreement / Difference
Meaning of Big-O	Describes the dominant growth rate as $n \rightarrow \infty$.	Describes asymptotic scaling and upper bounds.	Agreement
Actual execution time	Cannot be directly determined without constants and hardware context.	Dependent on constants, hardware, and specific implementations.	Agreement
Small inputs	The $O(n^2)$ algorithm might win due to smaller constant overhead.	$O(n^2)$ can execute faster for small n due to lower setup costs.	Agreement
Large inputs	The $O(n \log n)$ algorithm will always eventually dominate.	$O(n \log n)$ scales significantly better for massive datasets.	Agreement
Role of constants	Abstracted away mathematically, but practically important.	Hidden in notation, but heavily dictate real-world crossover points.	Agreement
Final claim	Unjustified to say A should "always" be selected.	Disagrees; the word "always" is incorrect in software engineering.	Agreement

Did the AI introduce any consideration that your group had overlooked?

- Yes. The AI specifically mentioned CPU caching behavior and branch prediction, noting that simpler $O(n^2)$ algorithms (like nested loops over contiguous arrays) often have much higher cache hit rates than complex $O(n \log n)$ algorithms (like tree-based structures), which can drastically delay the crossover point.

Did it make any statement that requires qualification or experimental evidence?

- The AI stated that the $O(n^2)$ algorithm "will likely be faster for $n < 50$." This specific threshold requires experimental evidence, as the actual crossover point depends entirely on the hidden constant factors of the specific code, not a universal rule of $n = 50$.

Part G

Experimental Plan

- **Input sizes:** A logarithmic spread to capture scaling: $n = 10, 100, 1000, 10000, 100000$.
- **Characteristics of the input:** Test three distinct cases to account for data sensitivity: a purely randomized dataset, an already sorted dataset, and a reverse-sorted dataset.
- **How equivalent inputs will be provided:** A single master dataset will be generated in memory. A deep copy of this exact dataset will be passed to Algorithm A and Algorithm B to ensure completely identical starting states.
- **What will be measured:** Wall-clock execution time in microseconds (excluding data generation and copy time).
- **Number of measurements:** 100 independent executions for each input size and input type.
- **How measurements will be summarized:** The mean execution time and standard deviation will be calculated across the 100 runs.
- **What should remain unchanged:** The physical machine, background OS processes (as isolated as possible), memory state, programming language, and compiler optimization flags (e.g., -O3).

Predictions

- **Prediction 1 (Small inputs):** For $n = 10$ and $n = 100$, Algorithm A ($O(n^2)$) will likely have a lower execution time due to less complex control flow and smaller constant overhead.
- **Prediction 2 (Large inputs):** For $n = 100,000$, Algorithm B ($O(n \log n)$) will execute orders of magnitude faster than Algorithm A.
- **Prediction 3 (Crossover point):** Yes, there will be a distinct region (likely between $n = 100$ and $n = 1000$) where the relative performance flips, because the mathematical growth of n^2 will eventually overwhelm Algorithm A's initial low-overhead advantage.

Final Reflection

1. **Before this tutorial, what did the statement “Algorithm A has better Big-O complexity than Algorithm B” mean to you?**

Before this tutorial, the statement implied a universal, absolute rule: Algorithm A was fundamentally superior and would inherently execute faster than Algorithm B across all use cases. There was a strong tendency to view asymptotic notation as a direct proxy for real-world wall-clock time, assuming that a better complexity class automatically negated any implementation details, constant overheads, or system constraints, regardless of whether the input size was 10 or 10,000,000.

2. **After completing the tutorial, what can you legitimately conclude from such a statement?**

We can legitimately conclude that Algorithm A scales more efficiently as the dataset size grows toward infinity. It provides a mathematical guarantee that there exists some specific input size—the crossover point—beyond which Algorithm A will consistently outperform Algorithm B. Big-O accurately describes the limiting behavior and the fundamental shape of the resource growth curve. This remains critical for architectural capacity planning, ensuring that software systems do not degrade exponentially when subjected to massive, unexpected loads in production environments.

3. **What can you not conclude without additional information or experimental evidence?**

We absolutely cannot conclude actual execution times in seconds, nor can we determine which algorithm is faster for finite, small, or even average-sized inputs. Big-O intentionally strips away constant factors and architectural realities. Without empirical data, we cannot account for critical system-level behaviors like CPU cache locality, branch prediction successes, or the underlying overhead of memory allocation. For instance, an $O(n^2)$ algorithm iterating over a contiguous block of memory might leverage hardware caches so effectively that it vastly outperforms a theoretically superior $O(n \log n)$ algorithm that relies on heavy pointer manipulation and fragmented heap allocations.

4. **If theoretical analysis and experimental measurements appear to disagree, should one simply be rejected? What should you investigate first?**

Neither the theoretical model nor the empirical data should be instantly rejected. A discrepancy typically indicates that the mathematical model’s underlying assumptions (such as treating all memory accesses as constant $O(1)$ operations) are clashing with real-world machine constraints.

The first step is to investigate the execution environment: Is the operating system triggering expensive memory page faults? Are compiler optimizations, such as loop unrolling or vectorization, heavily favoring one implementation over the other? Furthermore, one must investigate the test data itself to see if it inadvertently triggered a worst-case algorithmic scenario, or if hidden costs inside standard library utility functions are skewing the wall-clock measurements.

Submission

Contribution Statement:

- **Gaurav Acharya:** Formulated the theoretical complexity proofs in Parts A-C, drafted the experimental design matrix in Part G, and formatted the final submission document.
- **Sandeep:** Executed the AI simulation and constructed the comparative table for Part F, analyzed the hardware vs. implementation factors in Part E, and drafted the Final Reflections.